

CMU · 14-455 | Database System (2019)

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

聚合

排序

并发控制

数据库

内存管理

查询规划

分组

树索引

索引构建

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击 课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2: 扫码点击**底部菜单栏**
称为 **AI 内容创作者?** 回复 [添砖加瓦]

Lecture #02: Advanced SQL

15-445/645 Database Systems (Fall 2019)

<https://15445.courses.cs.cmu.edu/fall2019/>

Carnegie Mellon University

Prof. Andy Pavlo

1 Relational Languages

- Edgar Codd published major paper on relational models in the early 1970s. He originally only defined the mathematical notation for how a DBMS could execute queries on a relational model DBMS.
- User only needs to specify the result that they want using a declarative language (i.e., SQL). The DBMS is responsible for determining the most efficient plan to produce that answer.
- Relational algebra is based on **sets** (unordered, no duplicates). SQL is based on **bags** (unordered, allows duplicates).

2 SQL History

- **SQL**: Structured Query Language
- IBM originally called it “SEQUEL”.
- Comprised of different classes of commands:
 1. Data Manipulation Language (DML): SELECT, INSERT, UPDATE, DELETE.
 2. Data Definition Language (DDL): Schema definition.
 3. Data Control Language (DCL): Security, access controls.
- SQL is not a dead language. It is being updated with new features every couple of years. SQL-92 is the minimum that a DBMS has to support in order to claim they support SQL. Each vendor follows the standard to a certain degree but there are many proprietary extensions.

3 Aggregates

- An aggregation function takes in a bag of tuples as its input and then produces a single scalar value as its output. Can only be used in SELECT output list.

Example: “Get # of students with a '@cs' login”. The following three queries are equivalent:

```
SELECT COUNT(*) FROM student WHERE login LIKE '@cs';
```

```
SELECT COUNT(login) FROM student WHERE login LIKE '@cs';
```

```
SELECT COUNT(1) FROM student WHERE login LIKE '@cs';
```

- Can use multiple aggregates within a single SELECT statement:

```
SELECT AVG(gpa), COUNT(sid)
FROM student WHERE login LIKE '@cs';
```

- Some aggregate functions support the DISTINCT keyword:

```
CREATE TABLE student (  
  sid INT PRIMARY KEY,  
  name VARCHAR(16),  
  login VARCHAR(32) UNIQUE,  
  age SMALLINT,  
  gpa FLOAT  
);  
  
CREATE TABLE course (  
  cid VARCHAR(32) PRIMARY KEY,  
  name VARCHAR(32) NOT NULL  
);  
  
CREATE TABLE enrolled (  
  sid INT REFERENCES student (sid),  
  cid VARCHAR(32) REFERENCES course (cid),  
  grade CHAR(1)  
);
```

Figure 1: Example database used for lecture

```
SELECT COUNT(DISTINCT login)  
FROM student WHERE login LIKE '%@cs';
```

- Output of other columns outside of an aggregate is undefined (e. cid is undefined below)

```
SELECT AVG(s.gpa), e.cid  
FROM enrolled AS e, student AS s  
WHERE e.sid = s.sid;
```

- Thus, other columns outside aggregate must be aggregated or used in a GROUP BY command:

```
SELECT AVG(s.gpa), e.cid  
FROM enrolled AS e, student AS s  
WHERE e.sid = s.sid  
GROUP BY e.cid;
```

- HAVING: Filters output results after aggregation. Like a WHERE clause for a GROUP BY

```
SELECT AVG(s.gpa) AS avg_gpa, e.cid  
FROM enrolled AS e, student AS s  
WHERE e.sid = s.sid  
GROUP BY e.cid  
HAVING avg_gpa > 3.9;
```

4 String Operations

- The SQL standard says that strings are **case sensitive** and **single-quotes only**.
- There are functions to manipulate strings that can be used in any part of a query.

- **Pattern Matching:** The LIKE keyword is used for string matching in predicates.
 - “%” matches any substrings (including empty).
 - “_” matches any one character.
- **Concatenation:** Two vertical bars (“||”) will concatenate two or more strings together into a single string.

5 Output Redirection

- Instead of having the result a query returned to the client (e.g., terminal), you can tell the DBMS to store the results into another table. You can then access this data in subsequent queries.
- **New Table:** Store the output of the query into a new (permanent) table.

```
SELECT DISTINCT cid INTO CourseIds FROM enrolled;
```

- **Existing Table:** Store the output of the query into a table that already exists in the database. The target table must have the same number of columns with the same types as the target table, but the name of the columns in the output query do not have to match.

```
INSERT INTO CourseIds (SELECT DISTINCT cid FROM enrolled);
```

6 Output Control

- **ORDER BY:** Since results SQL are unordered, you have to use the ORDER BY clause to impose a sort on tuples:

```
SELECT sid FROM enrolled WHERE cid = '15-721'  
ORDER BY grade DESC;
```

You can use multiple ORDER BY clauses to break ties or do more complex sorting:

```
SELECT sid FROM enrolled WHERE cid = '15-721'  
ORDER BY grade DESC, sid ASC;
```

You can also use any arbitrary expression in the ORDER BY clause:

```
SELECT sid FROM enrolled WHERE cid = '15-721'  
ORDER BY UPPER(grade) DESC, sid + 1 ASC;
```

- **LIMIT:** By default, the DBMS will return all of the tuples produced by the query. You can use the LIMIT clause to restrict the number of result tuples:

```
SELECT sid, name FROM student WHERE login LIKE '%@cs'  
LIMIT 10;
```

Can also provide an offset to return a range in the results:

```
SELECT sid, name FROM student WHERE login LIKE '%@cs'  
LIMIT 10 OFFSET 20;
```

Unless you use an ORDER BY clause with a LIMIT, the tuples in the result could be different on each invocation.

7 Nested Queries

- Invoke queries inside of other queries to execute more complex logic within a single query. The scope of outer query is included in inner query (i.e. inner query can access attributes from outer query), but not the other way around.
- Inner queries can appear (almost) anywhere in query:

1. SELECT Output Targets:

```
SELECT (SELECT 1) AS one FROM student;
```

2. FROM Clause:

```
SELECT name
  FROM student AS s, (SELECT sid FROM enrolled) AS e
 WHERE s.sid = e.sid;
```

3. WHERE Clause:

```
SELECT name FROM student
 WHERE sid IN ( SELECT sid FROM enrolled );
```

- Example: “Get the names of students that are enrolled in '15-445'.”

```
SELECT name FROM student
 WHERE sid IN ( SELECT sid FROM enrolled WHERE cid = '15-445' );
```

Note that sid has different scope depending on where it appears in the query.

- Nest Query Results Expressions:
 - ALL: Must satisfy expression for all rows in sub-query.
 - ANY: Must satisfy expression for at least one row in sub-query.
 - IN: Equivalent to =ANY().
 - EXISTS: At least one row is returned.

8 Window Functions

- Performs “moving” calculation across set of tuples. Like an aggregation but it still returns the original tuples.
- **Functions:** Can be any of the aggregation functions that we discussed above. Can also be a special window functions:
 1. ROW_NUMBER: The number of the current row.
 2. RANK: The order position of the current row.
- **Grouping:** The OVER clause specifies how to group together tuples when computing the window function. Use PARTITION BY to specify group.

```
SELECT cid, sid, ROW_NUMBER() OVER (PARTITION BY cid)
  FROM enrolled ORDER BY cid;
```

You can also put an ORDER BY within OVER to ensure a deterministic ordering of results even if database changes internally.

```
SELECT *, ROW_NUMBER() OVER (ORDER BY cid)
  FROM enrolled ORDER BY cid;
```

- Important: The DBMS computes RANK after the window function sorting, whereas it computes ROW_NUMBER before the sorting.

9 Common Table Expressions

- Common Table Expressions (CTEs) are an alternative to windows or nested queries to writing more complex queries. One can think of a CTE like a temporary table for just one query.
- The WITH clause binds the output of the inner query to a temporary result with that name.
Example: Generate a CTE called “cteName” that contains a single tuple with a single attribute set to “1”. The query at the bottom then just returns all the attributes from “cteName”.

```
WITH cteName AS (  
    SELECT 1  
)  
SELECT * FROM cteName;
```

- You can bind output columns to names before the AS:

```
WITH cteName (col1, col2) AS (  
    SELECT 1, 2  
)  
SELECT col1 + col2 FROM cteName;
```

- A single query can contain multiple CTE declarations:

```
WITH cte1 (col1) AS (  
    SELECT 1  
)  
cte2 (col2) AS (  
    SELECT 2  
)  
SELECT * FROM cte1, cte2;
```

- Adding the RECURSIVE keyword after WITH allows a CTE to reference itself.
Example: Print the sequence of numbers from 1 to 10.

```
WITH RECURSIVE cteSource (counter) AS (  
    (SELECT 1)  
    UNION  
    (SELECT counter + 1 FROM cteSource  
     WHERE counter < 10)  
)  
SELECT * FROM cteSource;
```

CMU · 14-455 | Database System (2019)

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

聚合

排序

并发控制

数据库

内存管理

查询规划

分组

树索引

索引构建

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2: 扫码点击**底部菜单栏**称为 **AI 内容创作者?** 回复 [添砖加瓦]